Capacitação Profissional em Microeletrônica | PADIS Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de Semicondutores

O Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de Semicondutores (PADIS)

Jacques Costa Brito
Rômulo da Silva Lira
Thiago dos Santos Alves
Vitor Carvalho dos Santos

RELATÓRIO – MÁQUINA DE ESTADO FINITA (FSM)

MANAUS - AM
15 / 03 / 2026

Executores: iRede INOVAÇÃO E TECNOLOGIA | Patrocinador: Multi

SUMÁRIO

1. INTRODUÇÃO **3**
2. OBJETIVOS **4**
3. MATERIAIS E EQUIPAMENTOS UTILIZADOS **4**
4. RESULTADOS EXPERIMENTAIS **4**
4.1. FSM Moore (4 estados) 4
4.2 FSM Mealy (3 estados) 5
4.3 Diagrama de Estados 5
4.4 Tabela de Estados 6
4.5 Codificação de Estados 7
4.6 Implementação em SystemVerilog 8
4.7 Testbenches 8
4.8 Simulação no EDA Playground 9
4.8.1 Execução 9
4.9 Resultados 9
4.10 Comparação Moore x Mealy 11
5. CONCLUSÕES **13**
6. REFERÊNCIAS BIBLIOGRÁFICAS **14**

1. INTRODUÇÃO

> Hoje em dia, o termo *digital* tornou-se parte do nosso vocabulário diário devido ao modo intenso pelo qual os circuitos digitais e as técnicas digitais passaram a ser utilizados em quase todas as áreas: computadores, automação, robôs, tecnologia e ciência médica, transportes, telecomunicações, entretenimento, exploração espacial, e assim por diante [1].
> Diagramas de tempos são usados para descrever a operação detalhada de um sistema digital. As transições entre o nível de baixa tensão (quando a chave é solta) e o de alta tensão (quando a chave é pressionada) neste diagrama de tempos são traçadas como linhas verticais bem definidas [1].

"Um **sistema digital** é uma combinação de dispositivos projetados para manipular informação lógica ou quantidades físicas representadas no formato digital; ou seja, as quantidades podem assumir apenas valores discretos [1]".

"Um **sistema analógico** contém dispositivos que manipulam quantidades físicas representadas na forma analógica. Em um sistema analógico, as quantidades físicas podem variar ao longo de uma faixa contínua de valores [1]".

> No sistema binário, há apenas dois símbolos ou valores possíveis para os dígitos: 0 e 1. Mesmo assim, também pode ser usado para representar qualquer quantidade que possa ser representada em decimal ou em outro sistema de numeração. Entretanto, é comum que o sistema binário use um número maior de dígitos para expressar determinado valor [1].

"A álgebra booleana não é usada apenas como instrumento de análise e simplificação de sistemas lógicos; é também uma valiosa ferramenta de projeto usada para que um circuito lógico produza uma relação entrada/saída [1]".

"As entradas são consideradas variáveis lógicas cujos níveis lógicos determinam, a qualquer momento, os níveis da(s) saída(s). Como os valores possíveis de uma variável são apenas dois, a álgebra booleana é mais fácil de ser manipulada que a álgebra convencional [1]".

"O termo **máquina de estado** se refere a um circuito que sequencia um conjunto de estados predeterminados controlados por um clock e outros sinais de entrada [1]".

> As duas variações são chamadas de **modelo Mealy** de circuito sequencial e **modelo Moore**. No Mealy, os sinais de saída também são controlados por sinais de entrada adicionais, enquanto o Moore não possui controle externo para os sinais de saída gerados. A saída do modelo Moore é função apenas do estado atual do flip-flop [1].

2. OBJETIVOS

O objetivo desta tarefa é projetar, implementar em SystemVerilog e simular uma Máquina de Estado Finita (FSM) que detecta a sequência de bits "101". Serão criadas duas versões desta FSM: uma utilizando o modelo Moore e outra utilizando o modelo Mealy.

3. MATERIAIS E EQUIPAMENTOS UTILIZADOS

Foram utilizados computadores com o sistema operacional Windows 10 e Windows 11.

4. RESULTADOS EXPERIMENTAIS

Projete uma Máquina de Estado Finita (FSM) síncrona que detecte a sequência de bits "101". A FSM receberá um bit de entrada (data_in) a cada ciclo de clock e deverá gerar uma saída (sequence_detected) de 1 bit, que será '1' quando a sequência "101" for detectada. A detecção da sequência deve permitir sobreposição. Ou seja, se a entrada for "10101", a saída deve indicar a detecção do "101" duas vezes (uma na terceira entrada e outra na quinta).

Especificações:

- Entrada: data_in (1 bit)
- Saída: sequence_detected (1 bit)
- Clock: clk
- Reset: rst_n (ativo baixo e assíncrono)
- Comportamento: A saída sequence_detected deve ser '1' quando asequência mais recente dos três bits de entrada for "101".

Para detectar “101” com sobreposição, a FSM precisa lembrar o histórico recente de bits.

4.1. FSM Moore (4 estados)

No Moore a saída depende apenas do estado. Por isso é necessário um estado dedicado em que a sequência "101" foi completada. A Tabela 1 abaixo ilustra essa configuração.

Tabela 1 – Máquina de estado Moore.

Estado	Significado	Saída
S_IDLE	Nenhum bit relevante recebido	0
S_1	Último bit recebido foi ‘1’	0
S_10	Últimos dois bits formam ‘10’	0
S_101	Sequência ‘101’ completa	1

Fonte: Autores (2026).

4.2 FSM Mealy (3 estados)

No Mealy a saída depende do estado e da entrada. A detecção pode ser sinalizada na transição em que estamos em "10" e chega '1', sem estado extra. A Tabela 2 abaixo ilustra essa configuração.

Tabela 2 – Máquina de estado Mealy.

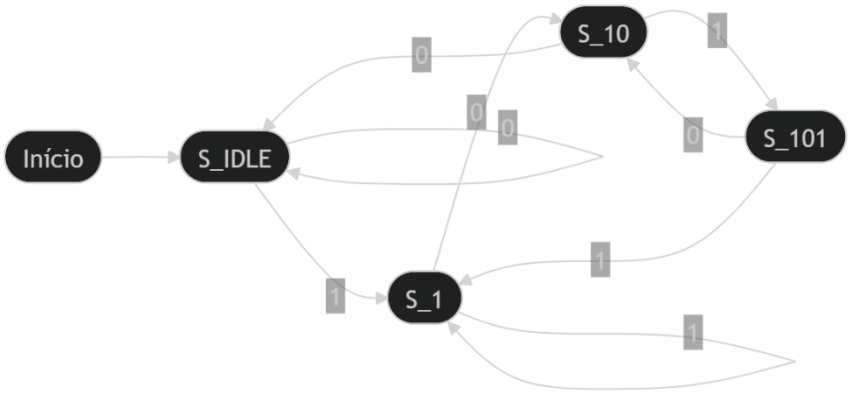
Estado	Significado	Saída
S_IDLE	Nenhum bit relevante	0
S_1	Último bit recebido ‘1’	0
S_10	Últimos dois bits formam ‘10’	1 somente se data_in =1

Fonte: Autores (2026).

4.3 Diagrama de Estados

No diagrama de estados Moore, a saída está associada aos estados (dentro dos círculos). Cada estado tem uma saída fixa. A Figura 1 a seguir ilustra esse caso.

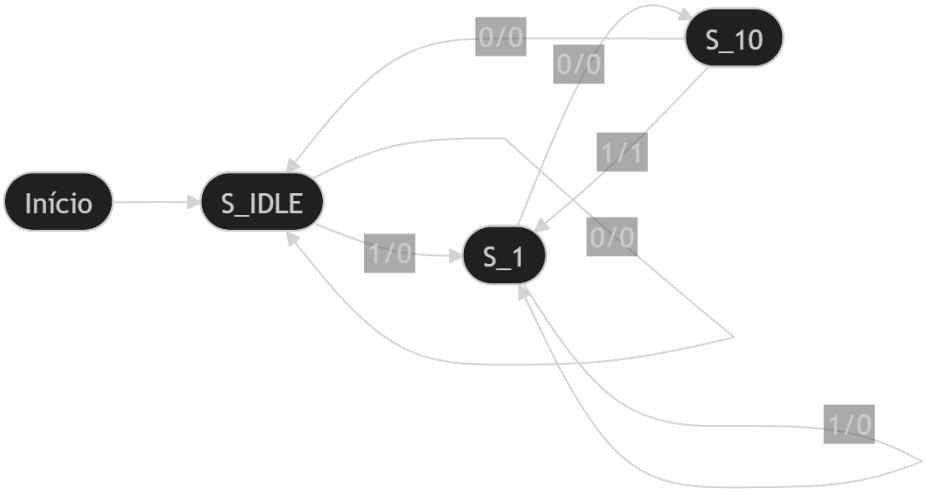
Figura 1 – Diagrama de estados Moore.



Fonte: Autores (2026).

No diagrama de estados Mealy, a saída está associada às transições (nos arcos), no formato entrada/saída. A Figura 2 a seguir ilustra esse caso.

Figura 2 – Diagrama de estados Mealy.



Fonte: Autores (2026).

4.4 Tabela de Estados

A tabela com estados da máquina Moore é ilustrada na Tabela 3 abaixo, enquanto que a tabela da máquina de Mealy é ilustrada na Tabela 4 também abaixo.

Tabela 3 – Máquina de estado Moore.

Estado	Entrada(data_in)	Próximo Estado	Saída(sequence_detected)
S_IDLE	0	S_IDLE	0
S_IDLE	1	S_1	0
S_1	0	S_10	0
S_1	1	S_1	0
S_10	0	S_IDLE	0
S_10	1	S_101	0
S_101	0	S_10	1
S_101	1	S_1	1

Fonte: Autores (2026).

Tabela 4 – Estados da máquina Mealy.

Estado	Entrada(data_in)	Próximo Estado	Saída(sequence_detected)
S_IDLE	0	S_IDLE	0
S_IDLE	1	S_1	0
S_1	0	S_10	0
S_1	1	S_1	0
S_10	0	S_IDLE	0
S_10	1	S_1	1

Fonte: Autores (2026).

4.5 Codificação de Estados

Foi utilizada **codificação binária** (sequencial) em ambas as FSMs:

- **Moore:** 4 estados → 2 bits (00, 01, 10, 11).
- **Mealy:** 3 estados → 2 bits (00, 01, 10).

Com poucos estados, a codificação binária minimiza o número de flip-flops (apenas 2). A codificação one-hot usaria 4 (Moore) ou 3 (Mealy) flip-flops, sem benefício relevante de timing ou área neste projeto.

4.6 Implementação em SystemVerilog

As duas FSMs foram implementadas no **estilo de três processos**:

1. **Processo 1 (always_ff):** sequenciador de estado – atualiza o estado no flanco do clock; reset assíncrono com negedge rst_n.
2. **Processo 2 (always_comb):** decodificador de próximo estado – unique case com valor default para evitar latches.
3. **Processo 3 (always_comb):** decodificador de saída – em Moore depende só do estado; em Mealy depende do estado **e** de data_in.

Foi adicionada a saída current_state (2 bits) para facilitar o monitoramento no testbench e no waveform, conforme sugerido no enunciado.

Para a FSM Moore (Sequence_detector_moore.sv), o módulo possui clk, rst_n, data_in, sequence_detected e current_state. Estados definidos com typedef enum logic [1:0]. A saída é 1 apenas quando o estado atual é S_101.

Para a FSM Mealy (Sequence_detector_mealy.sv), estrutura análoga, com 3 estados (sem S_101). O decodificador de saída usa current_state e data_in: só produz 1 quando está em s_10 e data_in == 1.

4.7 Testbenches

Foram criados tb_moore.sv e tb_mealy.sv, cada um:

- Instanciando o respectivo módulo (Moore ou Mealy).
- Gerando clock (período 10 ns) e reset (rst_n ativo baixo, liberado após ~15 ns).
- Aplicando a mesma sequência de entrada para permitir comparação direta: "11010010111".

Essa sequência inclui:

- Múltiplas ocorrências de "101" (com sobreposição).

- Sequências parciais ("1", "10").
- Bits que não completam "101".

Os testbenches usam uma task apply_stimulus para aplicar cada bit de forma síncrona (no negedge do clock) e contêm comentários indicando onde a sequência "101" **deve** ser detectada. Também utilizam $dumpfile e dumpvars para gerar arquivo VCD e $monitor para exibir no console: tempo, clk, rst_n, data_in, current_state e sequence_detected.

4.8 Simulação no EDA Playground

- Plataforma: EDA Playground.
- Linguagem: SystemVerilog/Verilog, modo Testbench + Design.
- Simulador utilizado: Aldec Riviera-PRO (ou outro de sua preferência).
- Arquivos: Na aba Design foi colado o código do detector (Moore ou Mealy); na aba Testbench, o código do testbench correspondente.
- Waveform: Marcada a opção "Open EPWave after run" para visualizar as formas de onda a partir do VCD gerado pelos testbenches.

4.8.1 Execução

1. **Simulação Moore:** Design = sequence_detector_moore.sv, Testbench = tb_moore.sv → Run.
2. **Simulação Mealy:** Design = sequence_detector_mealy.sv, Testbench = tb_mealy.sv → Run.

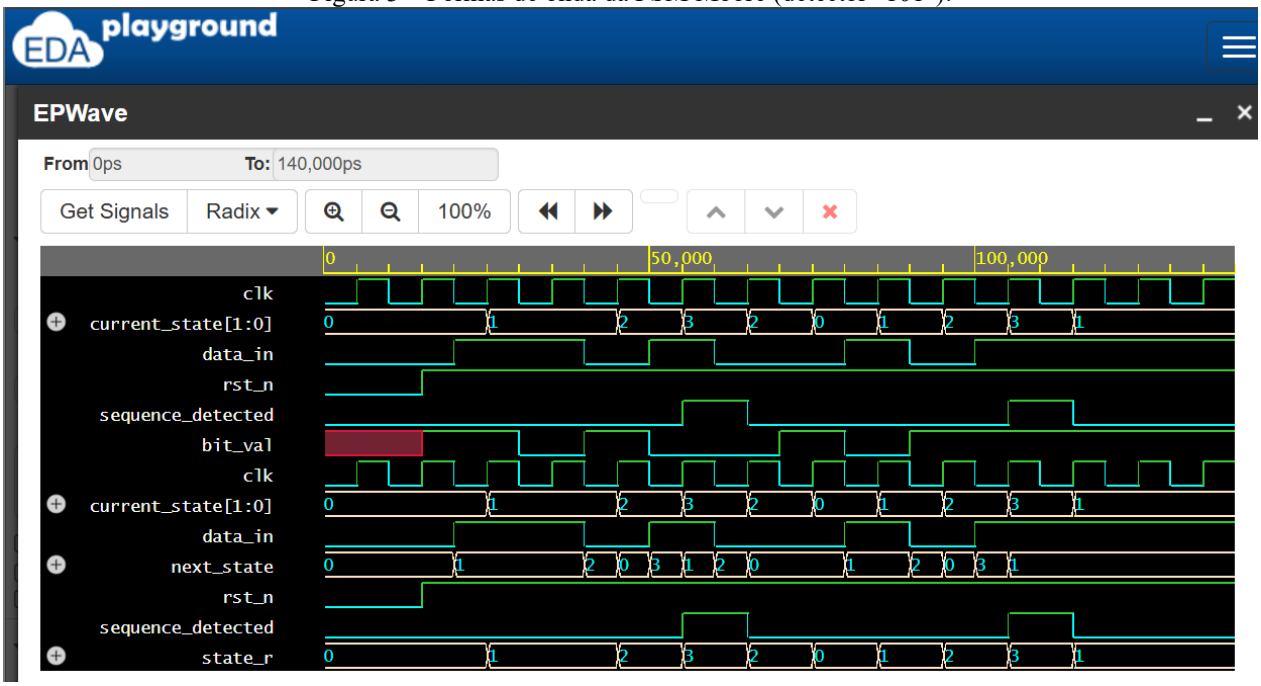
4.9 Resultados

A Figura 3 abaixo mostra o waveform da simulação da FSM Moore. Observe:

- rst_n inicia em 0 (reset) e depois sobe; a FSM passa a operar.
- data_in segue a sequência 11010010111.
- current_state assume os valores 0, 1, 2 e 3 (S_IDLE, S_1, S_10, S_101).

- sequence_detected vai a 1 nos dois instantes em que a sequência "101" é completada (por volta de 45 ns e 95 ns). No Moore, a saída só muda no ciclo seguinte ao bit que completa "101", quando a máquina já está no estado S_101 (estado 3).

Figura 3 – Formas de onda da FSM Moore (detector "101").



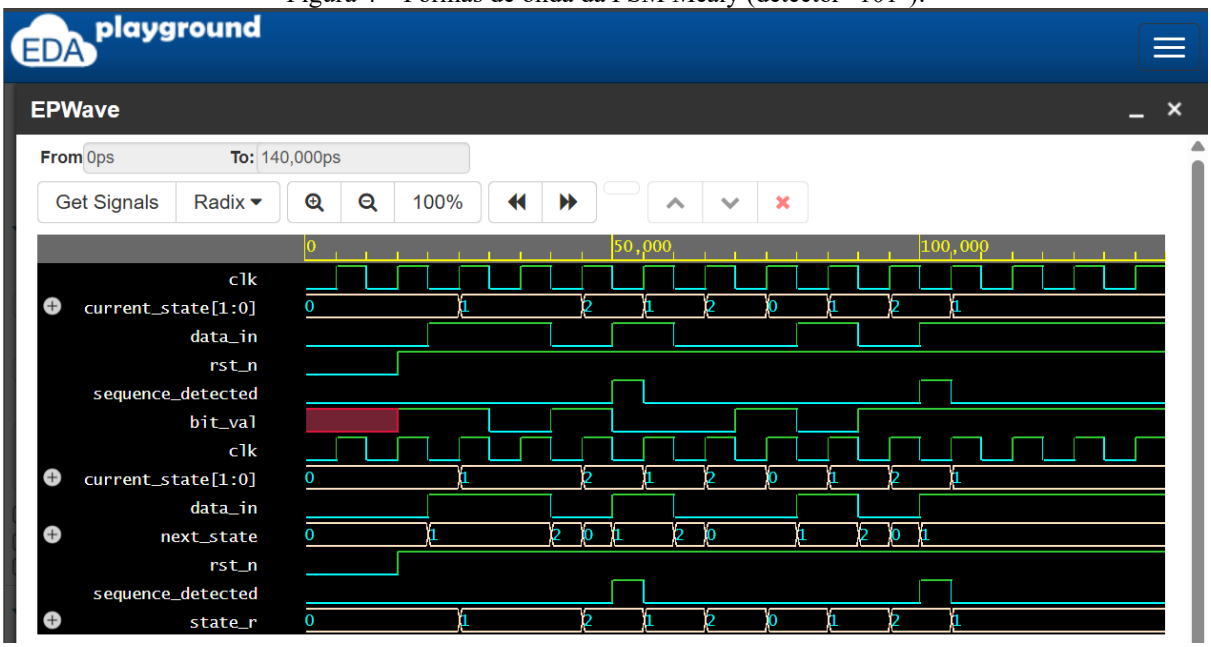
Fonte: Autores (2026).

No Moore a saída depende apenas do estado. Quando estamos em S_10 e recebemos o bit '1', no **mesmo** ciclo ainda estamos em S_10, então a saída continua 0. Só no **próximo** flanco de clock o estado muda para S_101 e aí sequence_detected vai a 1.

A Figura 4 abaixo mostra o waveform da simulação da FSM **Mealy**. Observe:

- Mesma sequência **data_in**: 11010010111 (e possível extensão no tempo).
- **current_state** usa apenas 0, 1 e 2 (três estados).
- **sequence_detected** vai a **1** nos mesmos instantes lógicos em que o "101" é completado, porém **no mesmo ciclo** em que o último '1' é aplicado. Isso ocorre porque no Mealy a saída é **combinacional**: depende do estado atual (S_10) e da entrada (1).

Figura 4 – Formas de onda da FSM Mealy (detector "101").



Fonte: Autores (2026).

No Mealy, quando a máquina está em S_10 e data_in é 1, o decodificador de saída já coloca sequence_detected no mesmo ciclo, sem esperar o próximo clock. Por isso a resposta é um ciclo mais “rápida” que no Moore.

4.10 Comparação Moore x Mealy

A Tabela 5 a seguir ilustra a comparação entre as máquinas de estados.

Tabela 5 – Comparação entre as márquinas de estados Moore e Mealy.

Aspecto	**Moore**	**Mealy**
Número de estados	4 (S_IDLE, S_1, S_10, S_101)	3 (S_IDLE, S_1, S_10)
Dependência da saída	Apenas estado	Estado + entrada
Instante em que sequence_detected vai a 1	Ciclo **seguinte** ao bit que completa "101"	**Mesmo** ciclo em que o '1' completa "101"
Uso de flip-flops	Maior (um estado a mais)	Menor

Fonte: Autores (2026).

Foi projetada e implementada em SystemVerilog uma FSM síncrona para detecção da sequência "101" com sobreposição, nas versões Moore e Mealy, utilizando:

- Estilo de três processos, typedef e uniquecase com default para evitar latches.
- Reset assíncrono ativo baixo.
- Testbenches com a mesma sequência de estímulos e geração de VCD para análise no EDA Playground.

As simulações confirmaram o comportamento esperado: detecção nos instantes corretos e a diferença temporal entre as duas máquinas (Moore com saída no ciclo seguinte; Mealy com saída no mesmo ciclo), em conformidade com o que foi solicitado na atividade.

5. CONCLUSÕES

O relatório em questão abordou o conceito de maquinas de estados finitas, sendo descritos os seus dois tipos. Esses conceitos são aplicados em circuitos sequenciais com o intuito de gerenciar o próximo estado com ou sem a interação do clock e de estados anteriores.

Cada máquina de estado foi elaborada com base nos requisitos solicitados, sendo descritas as características de cada uma delas, bem como os estados e as respectivas saídas. Após o projeto, foram realizadas as simulações no ambiente EDA Playground e então verificadas as formas de onda de cada máquina de estado. Com base nos requisitos de projeto que se deseja, tanto uma quanto outra pode ser utilizada.

6. REFERÊNCIAS BIBLIOGRÁFICAS

[1] TOCCI, Ronald J. Sistemas digitais : princípios e aplicações. 11. Ed. São Paulo: Pearson Prentice Hall, 2011.